

ENSC 251 group project proposal

Group members: Matthew Bennett, Jayden Chan, Arianna Suen

Game Description:

Contract Bridge is a card game played by four players divided into two teams: North-South and East-West. The game uses a standard 52-card deck, and each player receives 13 cards.

The goal of the game is for one team to win enough tricks to satisfy the contract made during the bidding phase. Each round has three main parts: dealing, bidding, and trick play.

During bidding, players take turns either passing or making a bid. A bid states how many tricks above six the team expects to win and which suit will be trump, or if the round will be played with no trump. The highest valid bid becomes the final contract.

During trick play, players each play one card per trick. Players must follow the suit that was led if they can. If they cannot follow suit, they may play any card. The highest card of the led suit wins unless a trump card is played, in which case the highest trump wins.

Minimum Viable Game:

- A standard 52-card deck.
- Four players.
- Two fixed partnerships: North-South and East-West.
- Random shuffling and dealing of 13 cards to each player.
- A simplified bidding phase.
- Contract selection based on the highest valid bid.
- Identification of declarer and dummy.
- A complete 13-trick play phase.
- Rule checking for following suit.
- Determination of the winner of each trick.
- Counting tricks won by each partnership.
- Determining whether the contract was made or defeated.
- Displaying the final result of the hand.

Planned Extensions:

Game Statistics

- Number of hands played.
- Contracts made.
- Contracts defeated.
- Tricks won by each partnership.

- Total score by team.

Multiple Rounds

- Instead of ending after one hand, the game may allow multiple deals and keep a running score between North-South and East-West.

Computer controlled players

Class Responsibilities:

Card Class

The Card class represents one playing card.

Information stored:

- Suit
- Rank

Responsibilities:

- Store the card's suit and rank.
- Return the card's suit.
- Return the card's rank.
- Convert the card to a printable string.
- Compare cards when needed.

Interacts with:

- Deck
- Hand
- Trick
- Player

Deck Class

The Deck class represents a standard 52-card deck.

Information stored:

- Dynamic or STL container of Card objects.
- Current card index for dealing.

Responsibilities:

- Create a full deck of 52 cards.
- Shuffle the deck.

- Deal cards to players.
- Reset the deck between hands.

Interacts with:

- Card
- Player
- Game

Hand Class

The Hand class stores the cards owned by a player.

Information stored:

- Collection of Card objects.

Responsibilities:

- Add cards to the hand.
- Remove a card when played.
- Display the hand.
- Sort cards by suit and rank.
- Check whether the hand contains a card of a required suit.
- Validate whether a selected card can legally be played.

Interacts with:

- Card
- Player
- Trick

Player Class

The Player class represents a player in the game.

Information stored:

- Player name.
- Player position: North, East, South, or West.
- Player hand.
- Team identifier.
- Whether the player is human or AI.

Responsibilities:

- Display the player's hand.

- Choose a bid.
- Choose a card to play.
- Track player position.
- Return player team information.

Interacts with:

- Hand
- Card
- Bid
- Game

Bid Class

The Bid class represents a bid made during the auction phase.

Information stored:

- Bid level.
- Contract suit or no trump.
- Whether the bid is a pass.

Responsibilities:

- Store bid information.
- Compare two bids to determine which is higher.
- Validate bid level and suit.
- Convert bid to a printable format.

Interacts with:

- BiddingManager
- Player
- Game

BiddingManager Class

The BiddingManager class controls the bidding phase.

Information stored:

- Current highest bid.
- Player who made the current highest bid.
- List of bids made.
- Number of consecutive passes.

Responsibilities:

- Ask players for bids in order.
- Validate each bid.
- Track the highest bid.
- Determine when bidding ends.
- Determine the final contract.
- Determine the declarer.

Interacts with:

- Bid
- Player
- Game

Contract Class

The Contract class stores the final contract after bidding.

Information stored:

- Contract level.
- Trump suit or no trump.
- Declaring team.
- Declarer.
- Dummy.
- Required number of tricks.

Responsibilities:

- Store final contract information.
- Calculate required tricks.
- Identify trump suit.
- Display contract information.

Interacts with:

- Bid
- Player
- Game
- ScoreManager

Trick Class

The Trick class represents one trick during the play phase.

Information stored:

- Cards played in the trick.
- Players who played each card.
- Lead suit.
- Trump suit.

Responsibilities:

- Add played cards.
- Track the lead suit.
- Determine whether a play follows suit.
- Determine the trick winner.
- Clear trick data after each trick.

Interacts with:

- Card
- Player
- Game

ScoreManager Class

The ScoreManager class handles scoring.

Information stored:

- North-South score.
- East-West score.
- Tricks won by each team.

Responsibilities:

- Count tricks won.
- Determine if the contract was made.
- Award points.
- Display the score.

Interacts with:

- Contract
- Player
- Game

Game Class

The Game class controls the overall program.

Information stored:

- Deck.
- Players.
- Current contract.
- Score manager.
- Current dealer.
- Current game state.

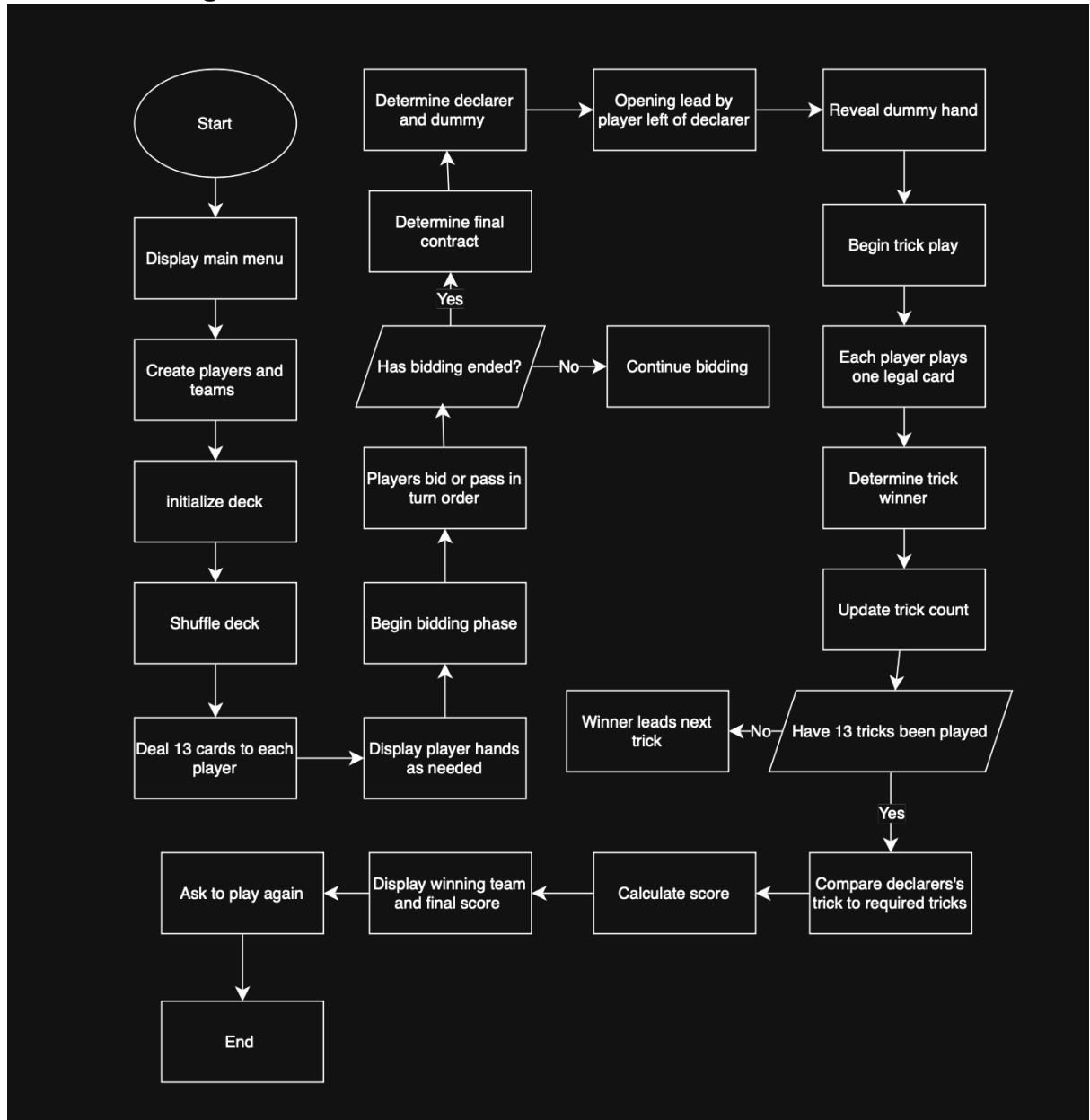
Responsibilities:

- Initialize the game.
- Create players.
- Shuffle and deal cards.
- Start bidding.
- Start trick play.
- Determine game result.
- Control turn order.
- End the game.

Interacts with:

- All major classes.

Game Flow Diagram:



Testing Plan:

Test Area	Test Case	Expected Result
Card	Create and display cards	Cards store correct rank/suit and display properly
Deck	Create full deck	Deck contains 52 unique cards
Deck	Shuffle and deal	Each of 4 players receives 13 cards
Hand	Add/remove cards	Hand size updates correctly
Hand	Check for suit	Program correctly detects if player can follow suit
Bidding	Valid bid entered	Bid is accepted if higher than current bid
Bidding	Invalid bid entered	Program rejects lower bids, invalid suits, or invalid levels

Bidding	Three passes after a bid	Auction ends and final contract is set
Contract	Calculate required tricks	Required tricks = bid level + 6
Contract	Determine declarer/dummy	Correct player becomes declarer and partner becomes dummy
Trick Play	Player follows suit	Legal card is accepted
Trick Play	Player breaks follow-suit rule	Illegal card is rejected
Trick Play	Determine trick winner	Highest trump wins, or highest led suit wins if no trump
Turn Order	Trick winner leads next	Game updates next starting player correctly
Scoring	Contract is made	Declarer's team receives credit/points
Scoring	Contract is defeated	Defending team receives credit/points
User Input	Invalid input entered	Program shows an error and asks again
Full Game	Complete one full hand	Game runs through 13 tricks and displays final result
Edge Case	All players pass	Program redeals or applies the chosen default rule
Edge Case	Player has no led suit	Player may legally play any card

Technical Challenges:

Challenge 1: Validating Bridge Bids

Bridge bidding has a strict order. A new bid must be higher than the previous bid, and the program must understand the ranking of suits and bid levels.

Why this is difficult:

The program must compare both the bid level and suit. For example, 1 Spade is higher than 1 Heart, but 2 Clubs is higher than 1 Spade.

Proposed solution:

We will create a Bid class with a comparison function such as `isHigherThan()`. This function will convert each bid into a comparable value based on level and suit ranking.

Challenge 2: Enforcing Follow-Suit Rules

During trick play, players must follow the suit led if they have a card of that suit.

Why this is difficult:

The program must check the player's entire hand before deciding whether a selected card is legal.

Proposed solution:

The Hand class will include a function such as `hasSuit(Suit suit)`. The Trick class will store the lead suit and ask the Handclass whether the player is required to follow suit.

Challenge 3: Determining Trick Winner

The winner of a trick depends on the led suit, the trump suit, and the ranks of the cards played.

Why this is difficult:

A card that is not the led suit normally cannot win unless it is a trump card. In no trump contracts, only the led suit matters.

Proposed solution:

The Trick class will have a function such as `determineWinner()`. This function will compare all played cards using the current trump suit and lead suit.

Team Responsibilities:

Jayden: card, deck, hand, player class

Arianna: trick, scoreManager, game class

Matthew: bid, biddingManager, contract class

All members will contribute to debugging, documentation, and final integration